

Linux Infrastructure 面试核心手册

Process • CPU • Memory • Filesystem • IO • Network • Namespace • cgroup
• Container • Troubleshooting

面向 Kubernetes / SRE / Infrastructure / Cloud Engineer 的面试复习资料

使用方式 不要背命令。每一章按“原理 -> 指标 -> 故障现象 -> 排查方法 -> 面试追问”复习，重点训练连续追问能力。

目录与复习优先级

章节	核心主题	优先级
1	Process / CPU / Scheduler / Load Average	★★★★★
2	Memory / Page Cache / OOM	★★★★★
3	Filesystem / inode / Page Cache / IO	★★★★★
4	Linux Network / TCP / conntrack	★★★★★
5	Namespace / cgroup / Container	★★★★★
6	K8s request/limit -> cgroup	★★★★★
7	cgroup v1 vs v2	★★★★☆
8	系统调用 / eBPF / 性能工具	★★★★☆
9	故障排查综合题	★★★★★
10	50 道高频面试题速查	★★★★★

推荐路线 先掌握 1-6 章，再刷第 10 章；面试前最后一天只看每章的“追问链”和综合排障题。

1. Process 与 CPU 调度

Linux Process / CPU 调度与排查

从 runnable task 到 CPU 执行，再到 load average / context switch

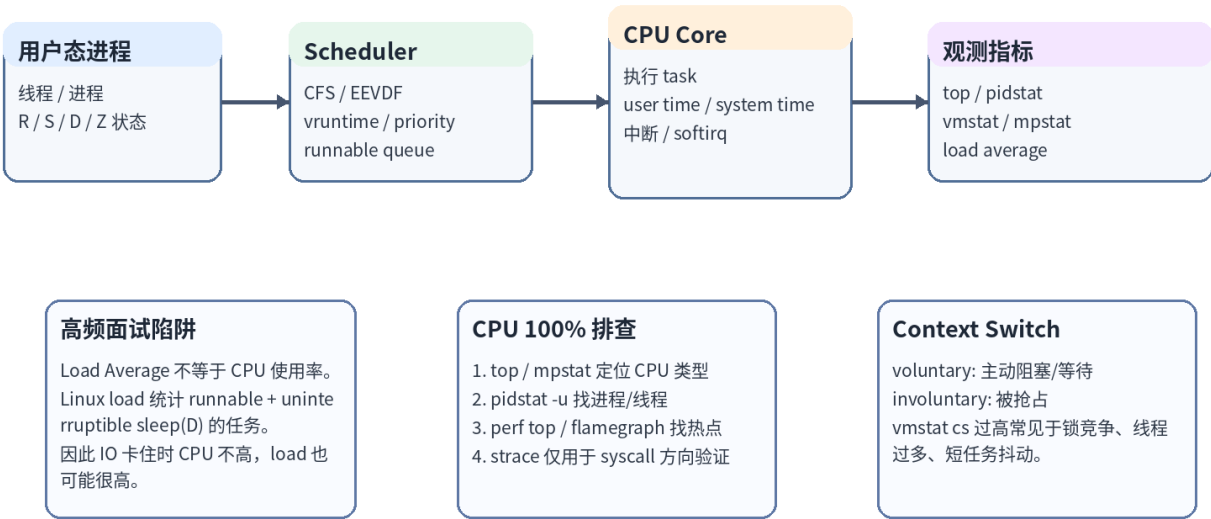


图 1: Linux Process / CPU 调度与性能排查主线

1.1 进程、线程与状态

Linux 内核调度的实体本质上是 task。传统语义上进程拥有独立地址空间，线程通常共享地址空间与文件描述符等资源，但在内核里都由 task_struct 表示。面试中要重点解释状态，而不是只背字母。

状态	含义	常见场景
R	Running / Runnable	正在 CPU 上执行或在 runqueue 等待
S	Interruptible Sleep	等待 socket、timer、condition 等，可被信号唤醒
D	Uninterruptible Sleep	常见于块 IO/内核等待；也计入 load average
Z	Zombie	子进程已退出，父进程尚未 wait 回收退出状态
T	Stopped/Traced	SIGSTOP、调试器 ptrace

1.2 Load Average 为什么不等于 CPU 使用率？

Linux load average 反映一段时间内 runnable task 与 uninterruptible sleep task 的平均数量。因此高 load 可能来自 CPU runqueue，也可能来自大量 D 状态任务，例如底层磁盘或网络存储卡住。

高频追问 “CPU 只有 30%，但 load 100，为什么？” - 优先检查 blocked/D state、IO latency、锁等待和 CPU 配额，而不是直接判断 CPU 不够。

1.3 CPU 100% 的标准排查路径

1. top/mpstat：先区分 user/system/iowait/softirq/steal。
2. pidstat -u -t：定位具体进程和线程。
3. perf top / perf record：判断用户态热点、内核热点、锁竞争或 cache miss。
4. 若 system time 高，再结合 strace、perf trace、eBPF 判断 syscall 或内核路径。

2. Memory：虚拟内存、Page Cache 与 OOM

Linux Memory：虚拟内存、Page Cache 与 OOM

面试必须区分 virtual address、RSS、cache、reclaim 与 cgroup OOM

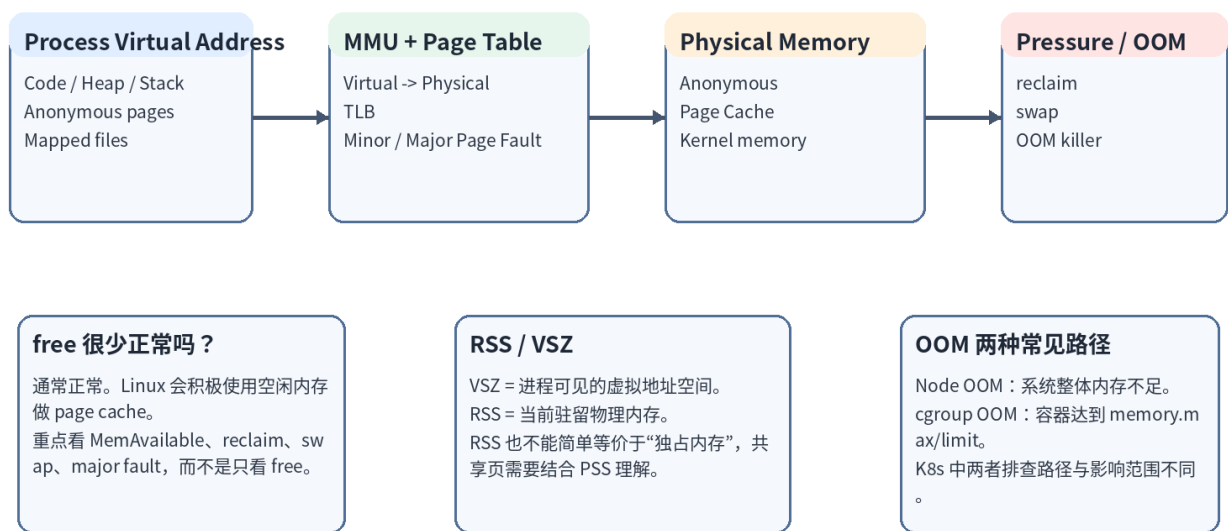


图 2: Linux Memory 从虚拟地址到 OOM 的完整逻辑

2.1 RSS、VSZ、PSS 与 Page Cache

指标	含义	注意点
VSZ/VIRT	虚拟地址空间大小	可包含尚未实际占用物理页的映射
RSS	驻留物理内存	包含共享页，不等价于独占内存
PSS	按共享比例分摊后的物理内存	分析多进程共享库更有意义
Page Cache	文件数据缓存	会消耗“已使用内存”，但通常可回收
MemAvailable	内核估算可供新 workload 使用的内存	比 free 更适合判断是否真的紧张

2.2 Page Fault

Minor fault 通常不需要从磁盘读入页面，例如页已在内存中但尚未建立当前进程的页表映射；Major fault 需要发生 IO，代价更高。面试中看到 major fault 飙升，要联系 page cache miss、swap、内存压力和存储延迟。

2.3 OOM：Node OOM 与 cgroup OOM

- Node OOM：主机整体内存严重不足，内核 OOM killer 在更大范围内选择牺牲进程。
- cgroup OOM：进程所在 cgroup 达到 memory.max（或 v1 limit），影响范围通常限制在该 cgroup。
- Kubernetes 中容器 OOMKilled 常见于 limit 太低或 workload 峰值内存超过 limit；Node MemoryPressure 则是另一个层面的资源问题。

3. Filesystem 、inode 、Page Cache 与 Block IO

Linux Filesystem / IO 数据路径

read/write 从用户态到 VFS、Page Cache、Block Layer、Disk

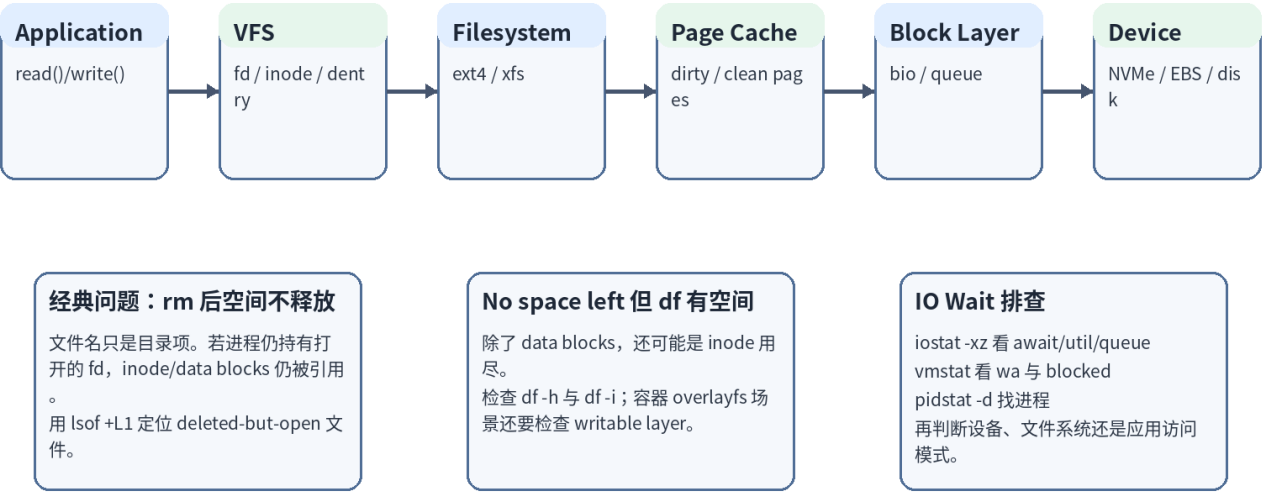


图 3: read/write 从应用到存储设备的数据路径

3.1 fd 、inode 、dentry 的关系

文件描述符 fd 是进程级句柄；dentry 负责目录项/路径名到 inode 的关联；inode 保存文件元数据并关联数据块。删除文件通常只是减少目录项和 inode 引用计数，若进程仍持有 fd，磁盘空间不会立即释放。

3.2 两个最经典的存储问题

现象	原因	排查
df -h 还有空间但创建文件失败	inode 用尽	df -i；定位海量小文件目录
rm 大文件后空间不下降	进程仍持有 deleted file fd	ls -lsof +L1；重启/关闭对应进程或 fd
iowait 高	设备/存储排队或同步 IO	iostat -xz、pidstat -d、vmstat
磁盘 util 高但吞吐不高	小 IO、随机 IO、队列深度或设备限制	看 await、r/s、w/s、avgqu-sz 等

4. Linux Network 与 TCP

Linux Network / TCP 数据路径与连接状态

从 socket 到 TCP/IP、netfilter、NIC，并串联 TIME_WAIT / CLOSE_WAIT / conntrack

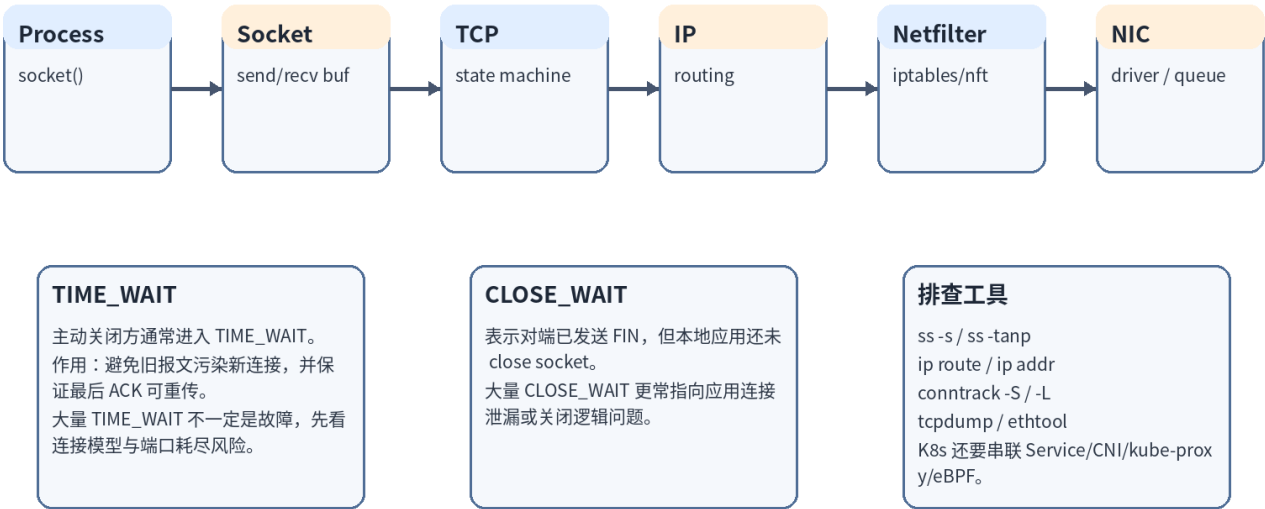


图 4: Linux 网络数据路径与 TCP 状态排查

4.1 TIME_WAIT 与 CLOSE_WAIT

状态	谁常进入	说明
TIME_WAIT	主动关闭连接的一方	等待旧报文过期并允许最后 ACK 重传；大量出现不一定是 bug
CLOSE_WAIT	收到对端 FIN 的一方	应用尚未 close socket；持续累积常提示连接泄漏
SYN_SENT	客户端	发出 SYN 等待 SYN/ACK；大量可能是网络不通/服务不可达
SYN_RECV	服务端	收到 SYN，等待 ACK；异常多可关注 backlog/SYN flood

4.2 连接超时怎么排查

5. DNS：名称是否解析正确。
6. Route：ip route 是否命中正确下一跳。
7. L3/L4：tcpdump 看 SYN 是否发出、SYN/ACK 是否返回。
8. Host firewall / netfilter：iptables/nftables/eBPF policy。
9. Conntrack：是否表满、drop、INVALID。
10. Kubernetes：继续检查 Service、EndpointSlice、CNI、kube-proxy/eBPF dataplane。

5. Namespace、cgroup 与 Container

Container 本质：Namespace + cgroup + rootfs

隔离“看见什么”，限制“能用多少”，文件系统提供“看到的根目录”

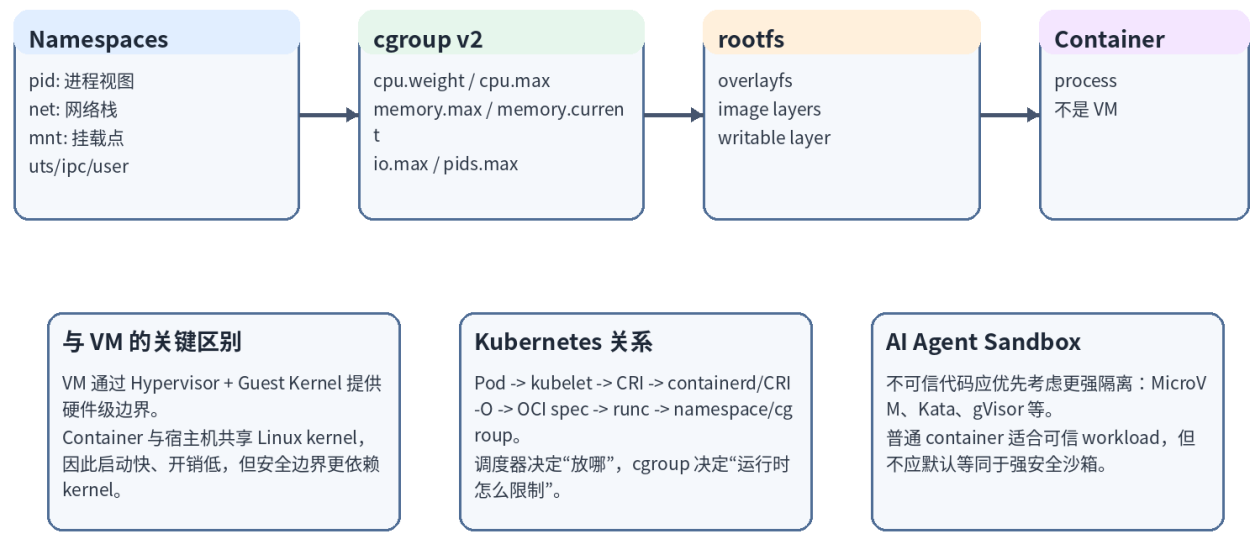


图 5: Container = Namespace + cgroup + rootfs

5.1 Namespace 管“看见什么”

Namespace	隔离内容	K8s/容器意义
pid	进程 ID 视图	容器内看到独立 PID tree
net	网卡、路由、端口、协议栈	Pod network namespace
mnt	mount table	独立 rootfs/volume mount
uts	hostname/domainname	容器 hostname
ipc	System V IPC / POSIX message queue	IPC 隔离
user	UID/GID 映射	rootless 容器与权限隔离的重要基础

5.2 cgroup 管“能用多少”

- CPU: weight 是竞争权重; max/quota 是硬上限。
- Memory: max 是硬上限; current 观测当前使用; events/pressure 用于判断 OOM 与压力。
- IO: weight/max 控制块设备带宽或 IOPS。
- PIDs: pids.max 防止 fork bomb。

6. Kubernetes requests/limits 如何落到 Linux cgroup

K8s resources requests/limits 与 cgroup (v1/v2) 控制文件 映射对照表

资源类型	K8s 字段		作用 / 含义	cgroup v1 控制文件		cgroup v2 控制文件		备注 / 额外说明
	requests	limits		对应文件	说明	对应文件	说明	
CPU	cpu: 500m	cpu: "2"	竞争时的相对权重 (CPU Share)	cpu.shares	控制 CPU 竞争时的分配比例 1 CPU = 1024 shares (默认) 最小值为 2	cpu.weight	1 ~ 10000, 默认 100 Nice 值映射: weight = (1.25^(20-nice))-100	不限制最大使用量, 只在竞争时生效
			CPU 最大使用上限 (CPU Quota)	cpu.cfs_quota_us cpu.cfs_period_us	配额/周期控制 quota/period = 最大 CPU 数 默认 period = 100000us(100ms)	cpu.max	格式: <max> <period> 例如: 200000 100000 => 2 CPU max 为 "max" 表示不限制	达到上限后会触发 CPU Throttling
Memory	memory: 512Mi	memory: 1Gi	内存保障 / QoS (非硬限制)	(通常无直接映射)	K8s 仅用于调度与 QoS 分类 在 v2 中可映射到 memory.min/low	memory.min (memory.low)	保障性内存 (根据 MemoryQoS 等) 防止被主动回收, 尽力保障	依赖 MemoryQoS 特性 Gate & 节点内核支持
			内存最大使用上限 (硬限制)	memory.limit_in_bytes	硬限制, 超过则触发 reclaim / OOM 等于 -1 表示不限制	memory.max	硬限制, 超过则触发 reclaim / OOM max 表示不限制	OOM 时可能被内核或 kubelet 杀死
IO - Block (磁盘块 IO)	(不支持)	(不支持)	限制块设备 IO 带宽或 IOPS (需扩展)	blkio.weight blkio.throttle.read_iops_device blkio.throttle.write_iops_device blkio.throttle.read_bps_device blkio.throttle.write_bps_device	权重: 10 ~ 1000 限制 IOPS / BPS (per device) 需要指定设备 major:minor	io.weight io.max (带设备号)	io.weight: 1 ~ 10000 io.max: <rbps> <wbps> <riops> <wiops> 例如: 8:0 rbps=10485760 wbps=10485760 riops=1000 wiops=1000	需 cgroup v2 & 内核 5.4+, 并显式配置设备号
IO - Throttle (延迟限制)	(不支持)	(不支持)	限制 IO 延迟	blkio.throttle.read_latency_us blkio.throttle.write_latency_us	对读/写延迟的最大允许值 (usec)	io.latency (带设备号)	格式: <device> <target_usec> <max_usec> 例如: 8:0 15000 30000	较少使用
PIDs	(不支持)	pids: "512"	进程数量上限	pids.max	-1 表示不限制	pids.max	max 表示不限制	防止 fork 炸弹
HugeTLB	hugepages-2Mi: 64M	hugepages-2Mi: 128Mi	HugePages 使用量	hugetlb.2Mi.limit_in_bytes hugetlb.2Mi.usage_in_bytes	分别为限制和使用量	hugetlb.2Mi.max hugetlb.2Mi.current	max: 限制 current: 当前使用	按分页大小分别控制
Devices	(不支持)	(不支持)	设备访问控制	devices.allow devices.deny devices.list	白名单 / 黑名单机制	devices.allow devices.deny devices.list	同 v1 (目前语义一致)	由运行时默认配置, 一般不直接修改
Network (net_cls)	(不支持)	(不支持)	网络流量分类标记	net_cls.classid	用于 tc / iptables 等流量分类	(已移除)	在 v2 中由其他机制替代 (如 eBPF / tc clsact)	较少使用
其它	-	-	冻结 / 杀死 / 统计等	freezer.state cpuset.cpus cpuset.mems cgroup.procs, cgroup.events	冻结状态 / CPU 亲和性 / 内存亲和性 / 事件统计等	cgroup.freeze cpuset.cpus.effective cgroup.events	冻结 / 亲和性 / 事件统计等	多用于运行时管理

注: IO 类控制需要内核、cgroup 版本、运行时、存储驱动都支持; 容器运行时 (containerd/cri-o) 通常通过 OCI Spec 将上述映射到底层 cgroup 文件。

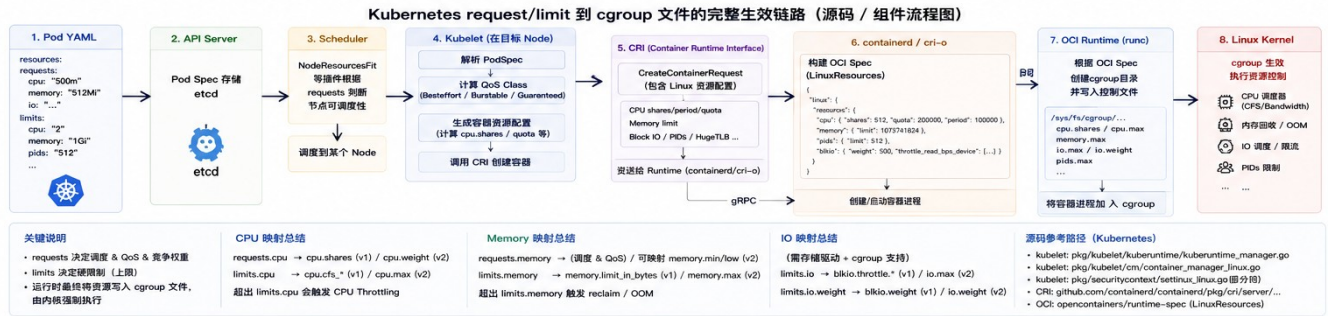


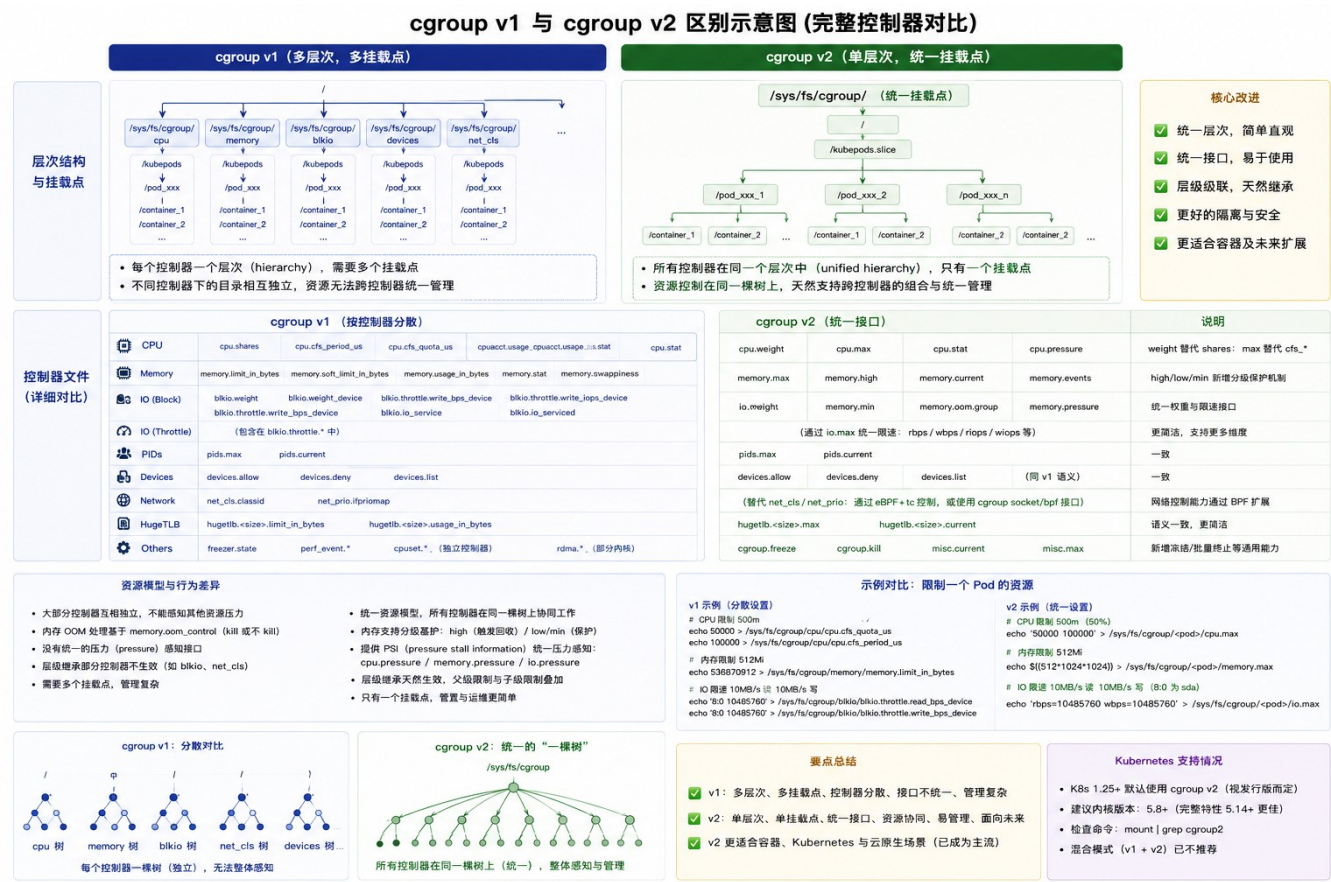
图 6: K8s requests/limits -> kubelet -> CRI -> OCI -> runc -> cgroup

6.1 对应关系速查

K8s 字段	cgroup v1	cgroup v2	含义
requests.cpu	cpu.shares	cpu.weight	CPU 竞争时相对权重
limits.cpu	cpu.cfs_quota_us + cpu.cfs_period_us	cpu.max	CPU bandwidth 硬上限/节流
requests.memory	无简单一一硬限制映射	可参与 memory.min/low (取决于机制)	调度保障/QoS 语义为主
limits.memory	memory.limit_in_bytes	memory.max	内存硬上限, 超过可能 cgroup OOM
PID 限制	pids.max	pids.max	限制进程数量
IO 限制	blkio.*	io.weight / io.max	块 IO 权重/限速

必须会讲 requests.cpu != “只能用这么多 CPU”。request 主要影响调度与 CPU 竞争权重; limit 才通过 CFS bandwidth/cpu.max 形成最大 CPU 使用上限。

7. cgroup v1 与 v2



工具	适合回答什么	注意点
strace	进程在调用哪些 syscall，哪里阻塞/报错	高频 syscall 场景开销可能明显，不宜无脑长时间开启
perf	CPU hotspot、kernel/userspace stack、硬件计数器	适合 CPU/锁/cache 等性能分析
bpftrace/eBPF	动态跟踪 kernel/user events、低侵入可编程观测	需要理解 hook 点与数据语义
ftrace	内核函数/事件跟踪	更偏内核调试
sar	历史系统指标	适合回看时间窗口

9. Linux 综合故障排查方法

Linux 性能问题：USE + RED 排查路径

先定位资源瓶颈，再定位进程/线程，再进入 syscall / kernel / network 细节

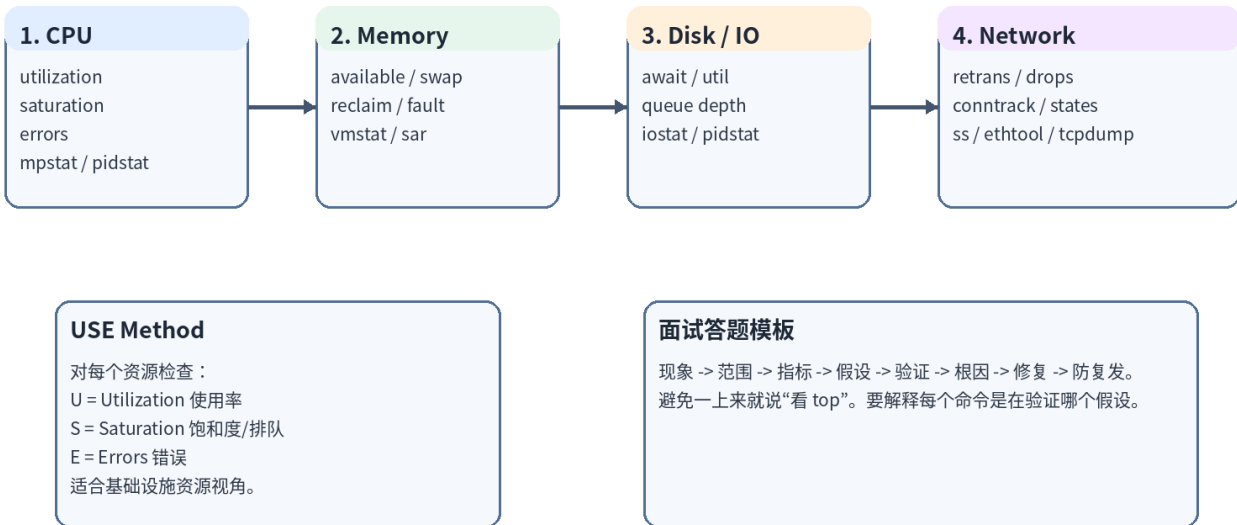


图 8: USE + 分层排障方法

9.1 面试题不要从“看 top”开始

优秀的回答需要体现推理结构。先确认影响范围和时间窗口，再按资源层收集证据，提出假设，然后用命令验证。命令是证据采集工具，而不是答案本身。

场景	第一层判断	关键工具
Latency 突增	CPU/Memory/IO/Network 哪一层饱和	vmstat, mpstat, iostat, ss
Load 高 CPU 不高	D state / IO / 锁 / cgroup throttling	ps, vmstat, iostat, pressure stall
Pod OOMKilled	cgroup limit vs Node memory pressure	kubectll, memory.events, dmesg
Pod CPU latency	throttling / runqueue / steal	cpu.stat, mpstat, pidstat
连接超时	DNS -> route -> SYN -> firewall -> service endpoint	dig, ip route, tcpdump, ss

磁盘满	blocks vs inode vs deleted-open	df -h, df -i, lsof +L1
-----	---------------------------------	------------------------

10. 50 道 Linux 高频面试题速查

1. 为什么 load average 高但 CPU 使用率不高？

load 还统计 D 状态任务；重点排查 IO/内核等待。

2. Zombie 是什么？为什么 kill -9 无效？

进程已经退出，只剩退出状态等待父进程 wait；需要处理父进程。

3. 什么是 context switch？

CPU 从一个 task 切换到另一个 task；涉及寄存器/调度上下文等切换。

4. CPU request 和 limit 区别？

request 影响调度与竞争权重；limit 形成 CPU bandwidth 上限。

5. 为什么 Node 很空容器仍 CPU throttling？

容器达到了自身 cgroup quota/cpu.max，与 Node 是否空闲无关。

6. free memory 很少是不是内存不足？

不一定；Linux 会用内存做 cache，应看 MemAvailable、reclaim、swap、PSI。

7. RSS 与 VSZ 区别？

RSS 是驻留物理页，VSZ 是虚拟地址空间。

8. 什么是 major page fault？

缺页需要发生 IO 才能取得页面。

9. Page Cache 有什么作用？

缓存文件页，减少磁盘 IO；也参与 write-back。

10. 什么时候发生 OOM？

内存回收无法满足分配；可能是全局 OOM 或 cgroup OOM。

11. memory limit 在 cgroup v2 对应什么？

memory.max。

12. 文件删除后空间为何不释放？

仍有进程持有 fd，inode/data blocks 引用未归零。

13. inode 用完是什么表现？

有磁盘空间但无法创建新文件。

14. fd 是什么？

进程打开文件/socket 等内核对象的整数句柄。

15. VFS 作用是什么？

给不同文件系统提供统一接口与抽象。

16. iowait 100% 说明磁盘一定慢吗？

不能单独下结论，需要结合设备 util/await/queue 和 workload。

17. 什么是 dirty page？

已被修改但尚未写回后端存储的 page cache 页面。

18. fsync 做什么？

请求把相关文件数据/元数据持久化到稳定存储，语义依文件系统/设备而定。

19. TIME_WAIT 为什么存在？

处理迟到报文并保证最后 ACK 可重传。

20. CLOSE_WAIT 多通常意味着什么？

应用收到 FIN 后没有及时 close socket。

21. SYN_SENT 多说明什么？

客户端发 SYN 后未收到有效响应，可能网络/服务/策略问题。

22. listen backlog 是什么？

监听 socket 等待建立连接的队列容量相关机制。

23. conntrack 是什么？

内核连接跟踪，为 NAT/防火墙等提供状态。

24. MTU 不匹配会怎样？

可能出现分片/PMTU 问题，典型表现是小包正常大包失败。

25. TCP retransmission 怎么看？

ss/netstat/sar/tcpdump 等结合 RTT/drop 分析。

26. namespace 解决什么？

隔离资源视图。

27. cgroup 解决什么？

资源统计、控制和层级管理。

28. container 与 VM 最大区别？

container 共享宿主 kernel；VM 通常有独立 guest kernel。

29. PID namespace 如何工作？

同一 task 在不同 PID namespace 可见不同 PID。

30. user namespace 有什么价值？

把容器 UID/GID 映射到宿主非特权 ID，增强 rootless 隔离。

31. cgroup v1/v2 最大区别？

v2 采用统一层级与统一控制模型。

32. `cpu.shares` 是硬限制吗？

不是，是竞争时权重。

33. `cpu.max` 的 200000 100000 表示什么？

每 100ms 周期最多获得 200ms CPU 时间，即约 2 CPU。

34. `memory.high` 与 `memory.max` 区别？

high 用于压力/回收阈值，max 是硬上限。

35. `pids.max` 的用途？

限制进程/线程数，防止 fork bomb。

36. 什么是 `system call`？

用户态请求内核服务的入口。

37. `strace` 与 `perf` 怎么选？

syscall 行为看 `strace`；CPU/stack/hardware counter 看 `perf`。

38. eBPF 优势是什么？

可在内核 hook 点安全运行程序并高效采集/处理事件。

39. `softirq` 高意味着什么？

可能网络包处理等软中断工作量大，要继续定位具体类别。

40. `steal time` 是什么？

虚拟机等待 hypervisor 调度物理 CPU 的时间。

41. D 状态进程能 kill 吗？

信号会挂起等待，但任务通常要等不可中断内核操作返回后才能处理。

42. 如何查哪个线程占 CPU？

`top -H / pidstat -t / ps -L`，再结合 `perf`。

43. 如何查端口被谁占用？

`ss -ltnp` 或 `lsof -i`。

44. 如何查进程打开了哪些文件？

`lsof -p PID` 或 `/proc/PID/fd`。

45. `/proc` 是什么？

内核提供的虚拟文件系统，暴露进程和系统运行状态。

46. 为什么 `swap` 可能导致延迟抖动？

匿名页换入需要 IO，major fault 带来高尾延迟。

47. 怎么判断 CPU 是饱和还是单线程瓶颈？

看总体 CPU、per-CPU、runqueue、单线程 CPU、profiling。

48. 怎么判断网络问题是客户端还是服务端？

双端 tcpdump + socket state + SYN/ACK/RTT/retrans 证据。

49. K8s Pod Pending 与 Linux cgroup 有什么关系？

Pending 通常是调度阶段；cgroup 是 Pod 已到 Node 后运行时资源控制。

50. 面试排障题最重要的结构是什么？

现象->范围->指标->假设->验证->根因->修复->防复发。

最后一页：面试前 10 分钟速记

主题	一句话记忆
Load	runnable + D state；不等于 CPU%
CPU request	调度 + weight，不是硬上限
CPU limit	CFS quota / cpu.max，可导致 throttling
Memory	free 少不等于不足；看 available/reclaim/PSI
OOM	区分 global OOM 与 cgroup OOM
Filesystem	fd -> dentry/inode -> blocks；deleted-open 会占空间
IO	看 latency + queue + util，不只看 iowait
TCP	TIME_WAIT 常在主动关闭方；CLOSE_WAIT 常提示应用未 close
Container	Namespace 隔离视图 + cgroup 控资源 + rootfs
排障	现象 -> 范围 -> 指标 -> 假设 -> 验证

核心目标 面试中不要只报命令。解释“为什么看这个指标、它能排除什么、下一步根据什么结果分支”，才能体现高级 Infra/SRE 的排障能力。